

Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Routing (Yönlendirme) ve Forwarding (İletme) Ayrımı
 - Routing, bir router'ın kendisine gelen paketin bir sonraki adımda nereye gönderileceğine karar vermesi sürecidir; bu kararı veren yapıya routing algoritması denir.
 - Forwarding ise bu kararın uygulanması, paketin buffer'dan alınıp doğru çıkış hattına konulması işlemidir.
 - İlişki: Scheduler/Dispatcher benzetmesi — routing kararının alınması, forwarding ise işin gerçekleştirilmesidir.
- Optimality Principle (Optimum Olma İlkesi)
 - Bir router (örneğin B), tüm diğer router'lara (M gibi) en iyi yolları bulundurursa, bu yollar bir "sink tree" (kök ağacı) formundadır.
 - Optimality principle der ki: B'den M'ye giden en iyi yol $B \rightarrow C \rightarrow J \rightarrow N \rightarrow M$ ise, bu yol üzerindeki her alt yol da ($B \rightarrow C$, $C \rightarrow J$, $J \rightarrow N$, $N \rightarrow M$) kendi aralarında en iyi yoldur.
 - Her router kendisi kök olacak şekilde ayrı bir sink tree oluşturmalıdır; topolojide her router için ayrı ağaç vardır.
- Shortest Path (En Kısa Yol) Algoritması
 - Dijkstra algoritması kullanılır; her link negatif olmayan bir ağırlıkla (weight) ilişkilendirilir.
 - Ağırlık fiziksel mesafe (km) değildir; maliyet (cost) olarak düşünülmelidir — güvenlik, para, gecikme, hız gibi faktörler.
 - Bazen işi basitleştirmek için her linkin ağırlığı 1 kabul edilir (hop sayısı); bu durumda en kısa yol en az hop olan yoldur.
- Flooding (Taşkın) Algoritması
 - Bir düğümden gelen paketin geldiği yöne bakılmaksızın tüm bağlantılara gönderilmesidir; en güvenilir routing algoritmasıdır (en kötü koşullarda bile paket hedefe ulaşır).
 - Dezavantajı: aşırı yükleme yapar, en verimli değildir; broadcast storm (yayın fırtınası) oluşturabilir.
 - Optimize edilebilir: (1) paketin geldiği hatta geri gönderilmez, (2) düğümler daha önce gönderdikleri paketleri hatırlar ve tekrar göndermez (her pakete unique ID gerekir).
 - Çoğu akıllı routing algoritması kontrollü bir şekilde flooding ile başlar, sonra söndürülür.
- Distance Vector Routing (Mesafe Vektörü)
 - 1975'te tasarlanmış, dağıtık (distributed) bir algoritmadır; Bellman-Ford algoritması olarak da bilinir.
 - Her düğüm sadece komşularına olan mesafeyi bilir ve en iyi mesafeleri tüm komşularına reklam eder (advertise eder).
 - Merkezi (centralized) routing ile karşılaştırma: merkezi sistemde tek bir nokta tüm topolojiyi bilir, routing tabloları oluşturur ve dağıtır. Dezavantajı: topoloji büyükse ölçeklenmez, dağıtık yavaş tepki verir.
 - Dağıtık (distributed) yaklaşım: divide and conquer mantığı; her düğüm kendi bölgesini bilir. Dezavantaj: komşunun ötesini göremez, hata durumlarında toparlanma (recovery) zor olabilir.
 - Hoca, 1975'te tasarlanmış bir algoritma için "neden şöyle yapmamışlar" demek yerine, o dönemin koşullarını göz önünde bulundurmak gerektiğini vurgular.

Hocanın Özellikle Vurguladığı Kısımlar

- Algoritma tasarımındaki trade-off'lar
 - Bellman-Ford 1975'te tasarlanmıştır; o dönemin teknolojik sınırlamaları (sensör ağlar, IoT cihazları yok, basit network yapıları) bugün geçerli olmayabilir. Ancak günümüzde hâlâ bazı topolojilerde (örneğin ad hoc network'ler, sensör ağlar) uygulanabilir.
 - Algoritmaları değerlendirirken o dönemin kısıtlarını bilmek önemlidir; “şimdi şöyle yapsaydık daha iyi olurdu” demek kolaycılıktır.
- Sink tree'nin her düğüm için ayrı oluşturulması
 - Bu, sınavda veya uygulamada sıkça karıştırılan bir noktadır: “tek bir sink tree” değil, topolojideki her router/host için bir sink tree vardır. Bu gözden kaçırılırsa routing hesabı yanlış yapılır.

Kısa Tekrar Notları

- Routing: karar süreci / Forwarding: uygulama
- Optimality principle: alt yol da optimumdur
- Dijkstra: link ağırlıklarına göre en kısa yol
- Flooding: en güvenilir, en verimsiz; optimize edilebilir
- Distance Vector: distributed, Bellman-Ford, komşuya bağlı
- Shortest path'te ağırlık = mesafe DEĞİL, maliyet

Detaylı Açıklamalar

Dersin ana konusu routing algoritmalarıdır. Routing, network katmanının temel işlevlerinden biridir: bir paketin hangi yoldan gideceğine karar verilmesi. Bunun için önce kriterler belirlenir (maliyet, hız, güvenilirlik, adalet). Bu kriterler algoritmanın “iyi” bir yolu bulmasını sağlar.

Kısa yol algoritması, tek bir router için en iyi yolu bulur. Sink tree, o router'dan diğer tüm düğümlere giden en iyi yolları içerir. Her router'ın kendi sink tree'si farklıdır; bu yüzden “topolojide birden fazla sink tree vardır” demek önemlidir.

Dijkstra algoritması, veri yapıları derslerinde öğrenilen klasik graf algoritmasıdır. Network topolojisinde her linkin ağırlığı olur; amaç, kaynaktan hedefe toplam ağırlığı en düşük yolu bulmaktır. Negatif ağırlık yoktur. Pratikte, maliyet (para), güvenilirlik, gecikme, hata oranı gibi faktörler ağırlık olarak kullanılabilir. Hoca özellikle vurgular: “Ağırlığı kilometre olarak düşünmeyin, maliyet olarak düşünün.” Çünkü network'te fiziksel uzaklık çok önemli değildir; önemli olan linkin kullanım bedeli, kalitesi, politik durumu gibi faktörlerdir. Uydu haberleşmesinde “down link” (yukarıdan aşağı) ucuz, “up link” (aşağıdan yukarıya) çok pahalıdır; yani aynı linkin iki yönünde farklı ağırlıklar olabilir.

Flooding, en ilkel routing yöntemidir: gelen paket geldiği yön hariç tüm bağlantılara gönderilir. Avantajı, topoloji bilgisi gerektirmemesi ve topolojideki herhangi bir arıza/bağlantı kopması durumunda bile paketin hedefe ulaşmasıdır. Dezavantajı, aşırı yük ve bant genişliği israfıdır. Pratikte, küçük network'lerde veya özel durumlarda (örneğin acil yayın, ağ keşfi) hâlâ kullanılır. Optimize edilebilir: her paket bir sequence number alır, düğümler gördükleri paketleri bir süre hatırlar ve tekrar göndermez.

Distance Vector, klasik internet algoritmasıdır. Her düğüm (router) sadece komşularıyla iletişim kurar. Komşularına “bana şu hedefe şu mesafede ulaşılabilir” der. Zamanla tüm düğümler tüm hedeflere en iyi mesafeleri öğrenir. Dezavantajı, komşunun komşusu hakkında bilgi sahibi olmamasıdır; hata durumlarında yavaş iyileşir (“count to infinity” problemi).

Hoca, distance vector'ün eski bir algoritma olduğunu ve bugün hâlâ bazı yerlerde kullanıldığını vurgular. Sensör ağlar veya küçük ölçekli dinamik topolojiler için hâlâ geçerli bir seçenek olabilir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.